

C++ Ortamında Konteyner Servis Yapılandırılmalarının Güvenli Round-Trip Düzenlenmesi

Safe Round-Trip Editing of Container Service Configurations in C++

Esila Şahin^{*1}, Ömer Yıldız², Mehmet Berk Kartal³, Ahmet Batuhan Canlı⁴ (*: *sorumlu yazar*)

^{*}esila.sahin@stu.fsm.edu.tr, ORCID: 0009-0007-8858-9540

¹Yazılım Mühendisliği Bölümü, Fatih Sultan Mehmet Vakıf Üniversitesi, İstanbul, Türkiye

^{2,3,4}TÜBİTAK BİLGEM UEKAE, Kocaeli, Türkiye

Özet Docker Compose dosyaları, geliştiricilerin yanı sıra CI/CD hatları ve dağıtım araçları tarafından da programatik olarak değiştirilmektedir. Bu dosyaları genel amaçlı YAML belgeleri gibi ele alan ayrıştır–değiştir–serileştir döngüleri; kullanıcı tanımlı $x-*$ alanlarını, yeni Compose sürümleriyle gelen modellenmemiş özellikleri, kısa ve uzun sözdizimi biçimlerini ve YAML tipini etkileyen tırnak bilgisini kaybedebilir. İncelediğimiz C++ kütüphaneleri arasında, Compose alanları için sınıf tabanlı bir API sunarken bilinmeyen ve $x-*$ alanlarının korunmasını sistematik round-trip deneyleriyle doğrulayan bir kütüphaneye rastlanmamıştır. Bu bildiride, Compose alanlarına tip-farkında erişim sağlayan, değişiklikleri mevcut YAML düğüm ağacı üzerinde gerçekleştiren ve dosyaları atomik olarak kaydeden C++20 kütüphanesi ComposeMorph tanıtılmaktadır. Yaklaşım; 38 kontrollü ve GitHub’dan derlenen 647 gerçek dünya Compose dosyası üzerinde yürütülen yedi deneyle doğruluk, yapı koruma, değişiklik yerelliği, Docker Compose CLI doğrulaması ve performans açısından değerlendirilmiştir. Hedeflenen tek alanlık bir düzenlemede dosyalara eklenen 802 bilinmeyen özelliğin ve 1.203 $x-*$ alanının tamamı korunmuş; kaydetmenin taban gürültüsü düştüğünde ölçülen on özelliğin tamamında düzenlemenin kendi katkısı tek satırla sınırlı kalmıştır. Değerlendirme ayrıca yamll-cpp tabanlı serileştirmede sessiz bir tip değişimi kusuru ortaya çıkarmıştır: tırnaklı skalerler tırnaksız yazıldığında docker compose config çıktıların yalnızca %38,1’ini kabul ederken, ayrıştırıcının sakladığı tırnak bilgisini kullanan serileştirme başarı oranını %100’e (892/892) çıkarmıştır. Bu düzeltme ölçülebilir bir performans bedeli oluşturmamış; eşleşik ölçümde tırnak koruyan serileştirici yamll-cpp’nin kendi serileştiricisinden %4–18 daha hızlı çalışmış ve en büyük dosyalarda yükle–değiştir–kaydet toplamının medyanı yaklaşık 15,23 milisaniye olmuştur. Koruma metinsel değil yapısaldır: yorumlar ve boş satırlar korunmadığından, değişiklik yapılmadan kaydedilen dosyaların yalnızca %4,0’ü bayt düzeyinde aynı kalmıştır. Düzenlenen alt ağacı yeniden kuran ham yamll-cpp kodunun ölçülen hiçbir vakada servisin kardeş alanlarını koruyamaması, bu kazanımın API tasarımından kaynaklandığını göstermektedir. Tüm veri kümeleri, betikler ve ham sonuçlar tek komutla yeniden üretilebilecek biçimde yayımlanmıştır.

Anahtar Kelimeler: Docker Compose, YAML Round-Trip Düzenleme, Kod Olarak Yapılandırma, Yapı Koruma, C++

Abstract Docker Compose files are increasingly modified programmatically by CI/CD pipelines and deployment tools. Generic parse–modify–serialize cycles can discard user-defined $x-*$ fields, unmodeled properties introduced by newer Compose Specification versions, short- and long-syntax forms, and quoting information that affects YAML types. Among the C++ libraries reviewed, none combines a class-based API for Compose fields with preservation of unknown and $x-*$ properties validated through systematic round-trip experiments. We present

ComposeMorph, a C++20 library that provides type-aware access to Compose fields, edits the existing YAML node tree in place, and saves files atomically. We evaluate the approach in seven experiments over 38 controlled files and 647 real-world Compose files collected from GitHub, measuring correctness, structure preservation, change locality, Docker Compose CLI validation, and performance. All 802 injected unknown properties and all 1,203 $x-*$ fields survive a targeted single-property edit; after baseline save noise is subtracted, the edit itself is confined to one line for all ten measured properties. The evaluation also exposes a silent type-coercion defect in yaml-cpp-based serialization: when quoted scalars are emitted without quotes, docker compose config accepts only 38.1% of the outputs, whereas serialization driven by the quote information retained by the parser raises validation success to 100% (892/892). This correction introduces no measurable performance cost: in paired measurements, the quote-preserving serializer is 4–18% faster than yaml-cpp's serializer, and the median load–modify–save time is approximately 15.23 ms even for the largest files. Preservation is structural rather than textual: because comments and blank lines are not retained, only 4.0% of files are byte-identical after an unmodified save. A direct yaml-cpp implementation that rebuilds the edited subtree preserves the service's sibling fields in none of the measured cases, indicating that the observed preservation follows from the API design. All datasets, scripts, and raw results are published for one-command reproduction.

Keywords: *Docker Compose, YAML Round-Trip Editing, Configuration As Code, Structure Preservation, C++*

I. GİRİŞ

Docker Compose dosyaları, uygulamanın nasıl çalıştırılacağını tanımlayan teknik belgeler olmanın yanında geliştiriciler, CI/CD hatları ve dağıtım araçları arasında paylaşılan bir sözleşmedir [1, 2]. Otomatik bir aracın yaptığı küçük bir güncellemenin ilgisiz alanlara yansması, kod incelemesini güçleştirir ve çalışan bir yapılandırmayı fark edilmeden bozabilir. Bu nedenle programatik düzenlemede temel beklenti, yalnızca istenen değişikliğin yapılması ve dosyanın geri kalanının anlamının korunmasıdır.

Bu beklentiye karşılık, YAML'ı okuyup yeniden yazmaktan daha kapsamlı bir problemidir. Compose Specification zamanla genişler; aynı kavram farklı sözdizimleriyle ifade edilebilir ve düzenleyici, henüz tanımadığı alanlarla karşılaşabilir. Buna karşın C++ ekosisteminde, Compose kavramlarını doğrudan temsil eden ve bu çeşitliliği yeniden kullanılabilir bir arayüz arkasında yöneten ortak bir kütüphane bulunmamaktadır.

Bu çalışma, sınıf tabanlı bir Compose API'sinin mevcut belge ağacı üzerinde yerinde değişiklik yaparak bu gereksinimi karşılayıp karşılayamayacağını inceler. Bu amaçla geliştirilen ComposeMorph; hedef alanı düzenlerken belgenin modellenmeyen bölümlerini aynı ağaçta tutar ve kaydetme aşamasında YAML değerlerinin anlamını korumayı amaçlar. Yaklaşım, birbirini tamamlayan kontrollü ve gerçek dünya veri kümelerinde doğruluk, koruma, değişiklik yerelliği, ileriye dönük uyumluluk ve performans bakımından sınanmıştır.

Bu bildirinin katkıları şunlardır:

1. Compose alanları için sınıf tabanlı API, modellenmeyen alanlara genel erişim ve atomik kaydetme sunan yeniden kullanılabilir bir C++ kütüphanesi.
2. Modellenmeyen Compose ve $x-*$ alanlarını koruyan yerinde düzenleme yaklaşımının gerçek dünya dosyalarıyla değerlendirilmesi.
3. Sessiz tip değişimi ile kısa sözdiziminin istemeden mapping biçimine dönüşmesi sorunlarının belirlenmesi ve giderilmesi.

4. İki veri kümesini, yedi deneyi ve yeniden üretim betiklerini içeren açık değerlendirme paketi.

II. ALTYAPI VE MOTİVASYON

A. Docker Compose Specification

Compose Specification [1], bir uygulamanın servislerini ve bunların ağlarını, volume'larını, config ve secret tanımlarını YAML biçiminde tanımlar. Belgenin üst seviyesinde `services`, `networks`, `volumes`, `configs`, `secrets` gibi bölümler ile `x-` önekli extension fields (`x-*`) bulunur. Extension fields, spesifikasyonun araçlara ayırdığı resmî genişletme noktalarıdır: şema bunları serbest bırakır, ancak `x-` öneki taşımayan tanınmayan anahtarları reddeder. Spesifikasyon ayrıca aynı bilgiyi iki biçimde ifade etmeye izin verir. Örneğin portlar kısa sözdiziminde `"8080:80"` biçiminde bir dizi elemanı, uzun sözdiziminde ise `target/published` anahtarlarını taşıyan bir eşleme (mapping) olarak yazılabilir; `environment`, `labels`, `extra_hosts` ve `volumes` için de benzer ikilikler vardır. Bir düzenleme kütüphanesi her iki biçimi de bozmadan taşımak zorundadır.

B. YAML skalerleri ve örtük tip çözümlemesi

YAML'da bir skaler değerin tipi, tırnak kullanımına bağlıdır. Tırnaksız (plain) yazılmış `3.8` bir kayan noktalı sayı, `true` bir boolean, `8080` bir tam sayı olarak çözümlenir; aynı değerler tırnaklandığında string olur. YAML 1.1 çözümleme kuralları bu kümeyi `yes/no/on/off` gibi boolean eşdeğerlerini, onaltılık ve sekizlik tam sayıları, `1:30` biçimindeki altmışlık (sexagesimal) sayıları ve tarih/zaman damgalarını da içerecek biçimde genişletir. Dolayısıyla tırnak, YAML'da biçimsel bir tercih değil, anlam taşıyan bir işarettir: `version: "3.8"` ile `version: 3.8` farklı belgelerdir ve Compose şeması `version` alanının string olmasını şart koşar.

YAML ayrıca yorum satırları, boş satırlar, girinti tercihleri, `anchor/alias` (&ad, *ad) ve birleştirme anahtarı (<<) gibi, belgenin veri içeriğine değil sunumuna ait öğeler barındırır. Bu öğelerin bir kısmı ayrıştırma sırasında veri modeline hiç taşınmaz.

C. Round-trip düzenleme problemi

Bir yapılandırma dosyasını programatik olarak düzenlemenin temel akışı *ayrıştır* → *değiştir* → *serileştir* biçimindedir. Dosyanın bu döngü sonunda ne ölçüde korunacağı, ayrıştırıcı tarafından tutulan ara temsile bağlıdır. Somut sözdizimi ağacı (İng. Concrete Syntax Tree, CST) kullanan araçlar, kaynak metindeki sözdizimsel ayrıntıları da saklayabildiği için yalnızca değiştirilen bölümü yeniden yazabilir. ruamel.yaml [4] ve eemeli/yaml [5] bu yaklaşım sayesinde yorum ve biçim bilgisinin önemli bölümünü korur. yaml-cpp [3] ise belgeyi düzenlenebilir bir YAML düğüm ağacına dönüştürür. Bu model belge yapısını, anahtar sırasını ve değerleri korusa da yorum, boş satır, girinti ve bazı skaler gösterim ayrıntılarını tam olarak saklamaz. Bu nedenle dosya kaydedilirken belgenin tamamı yeniden serileştirilir ve hedeflenen değişikliğin dışındaki satırlarda da fark oluşabilir. Daha önemlisi, serileştirme sırasında tırnak bilgisinin kaybedilmesi bazı skalerlerin YAML tipinin sessizce değişmesine neden olabilir.

D. İleriye dönük uyumluluk

Compose Specification gelişmeye devam etmektedir. Bir düzenleme kütüphanesinin nesne modeli, spesifikasyonun her sürümünü takip edemez. Bu nedenle kütüphanenin bilmediği alanları *kaybetmemesi*, yani modellenmemiş her yapıyı olduğu gibi taşıması pratik bir gerekliliktir. Şema temelli (dataclass/Pydantic benzeri) yaklaşımlar bu noktada dezavantajlıdır: model, tanımadığı bir alanı saklayacak bir yere sahip değilse serileştirmede o alan kaybolur.

III. İLGİLİ ÇALIŞMALAR

Bu bölüm, Docker Compose yapılandırmalarının programatik olarak düzenlenmesiyle ilişkili araç ve kütüphaneleri karşılaştırmaktadır.

A. YAML kütüphaneleri

İncelenen C/C++ kütüphaneleri yorum ve biçim koruyan round-trip düzenlemeyi desteklenen bir özellik olarak belgelememektedir. yaml-cpp [3] yorumları atar ve ilgili talep 2015'ten beri açıktır [8,9]; libyaml yorum belirteci üretmez [10,11]. rapidyaml'da skaler tip sadakatini bozan ters yönlü bir tırnaklama hatasının daha önce giderilmiş olması [12,13], tırnak kararlarının genel bir hata kaynağı olduğunu göstermektedir. fkYAML da round-trip koruma iddiası taşımaz [14].

B. Yorum koruyan ve CST tabanlı araçlar

ruamel.yaml [4], eemeli/yaml [5] ve Rust yaml-edit [15] CST veya kayıpsız ağaçlarla yorum ve biçimi koruyabilir; benzer yaklaşım LibCST ve cstree'de görülür [16,17]. Ancak bu araçlar genel amaçlıdır, Compose nesne modeli sunmaz ve C++ değildir.

C. Kubernetes ve Docker Compose araçları

Kubernetes kyaml [7] yapıyı ve yorumları korumayı hedeflese de go-yaml kaynaklı yorum kayıpları bildirilmiştir [18]. Compose'un başvuru kütüphanesi compose-go [6] yükleme ve doğrulamaya odaklanır. fishfinger, compose-py ve docker-compose-yaml-parser programatik düzenleme sunar [19–21], ancak bilinmeyen alan ve x^* korumasını ölçülmüş bir garanti olarak belgelemez.

D. Ampirik çalışmalar

GitHub tabanlı çalışmalar Compose kullanımını ve kompozisyon örüntülerini incelemiştir [2,22]; YAML'ın model yönetimi dilleriyle işlenmesi de model düzeyinde ele alınmıştır [23]. Bu bildirinin odağı ise mevcut dosyanın en az yan etkiyle düzenlenmesidir.

E. Konumlandırma

Taramada, Compose'u C++ ile tipli bir API üzerinden düzenlerken modellenmeyen alanların korunmasını deneysel olarak değerlendiren bir kütüphaneye rastlanmamıştır. ComposeMorph bu boşluğu hedefler; yorum ve tam biçim koruyan bir CST düzenleyicisi değildir.

IV. TASARIM

A. Nesne modeli

Kütüphanenin giriş noktası ComposeFile sınıfıdır: bir dosyayı yükler, servislere ve üst seviye koleksiyonlara erişim verir, dosyayı kaydeder. `service(ad)` çağrısı bir Service nesnesi döndürür; Service de Compose kavramlarını doğrudan yansıtan alt nesneler sunar (Environment sınıfı labels için de kullanılır). Service üzerinden Environment, ExtraHosts, Ports, Volumes, Networks, BuildConfig, HealthCheck, DependsOn ve DeployConfig nesnelerine erişilir. ComposeFile ayrıca NetworkCollection, VolumeCollection, ConfigCollection ve SecretCollection üst seviye koleksiyonlarını sunar.

Bu sınıfların hiçbirisi veriyi kendi içinde kopyalamaz; her biri, altta yatan YAML düğüm ağacının ilgili düğümüne tutunan ince bir görünümdür. Bir Service nesnesi, servisin düğümüne bir tutamaçtır; üzerinde yapılan her değişiklik doğrudan ağaçta gerçekleşir. Böylece kütüphanenin modellemediği kardeş alanlar, düzenleme sırasında hiçbir zaman okunup yeniden yazılmaz.

B. Tipli API ile genel alan API'nin ilişkisi

Tipli API, spesifikasyonun sık kullanılan alanlarını adlandırılmış işlemlerle sunar; örneğin bir imaj etiketini güncellemek tek satırdır:

```
compose.service("api")
    .setImage("company/api:2.0");
```

Modellenmemiş her alan için aynı nesne, nokta gösterimli genel bir özellik API'si sağlar:

```
api.set("x-company-security.hsm", "enabled");
auto v = api.get<std::string>(
    "logging.options.max-file");
api.remove("deploy.placement.constraints");
```

İki API aynı ağaç üzerinde çalışır ve birbirinin yerine geçebilir: tipli işlemler, genel alan API'nin yapacağı düğüm yazma işleminin adlandırılmış ve tip kontrollü bir kısayoludur. Bu ilişki, ileriye dönük uyumluluğun kaynağıdır. Spesifikasyona yarın eklenecek bir alan için kütüphanenin güncellenmesi gerekmez; alan genel alan API ile okunup yazılabilir, hiç dokunulmasa bile dosyada aynen kalır.

C. Kısa ve uzun sözdizimi

Ortam değişkenleri, etiketler ve ek host kayıtları için kullanılan sınıflar, altlarındaki düğümün liste mi yoksa eşleme mi olduğunu çalışma zamanında tespit eder ve dosyanın mevcut biçimini korur. Örneğin ortam değişkenleri kısa sözdiziminde, yani KEY=value biçimindeki liste girdileri olarak yazılmışsa, bir değişkeni güncelleyen çağrı ilgili girdiyi yerinde değiştirir ya da listenin sonuna yeni bir girdi ekler; bölümü eşlemeye dönüştürmez. Ports, Volumes ve Networks yardımcıları da uzun sözdizimli (eşleme) girdileri hata vermeden işler.

D. Hata yönetimi ve güvenli kaydetme

Ayrıştırılmayan dosya, bulunamayan servis ve iş kuralı ihlali, her biri kendi istisna türüyle bildirilir; örneğin olmayan bir servise erişim ServiceNotFoundException fırlatır. Kaydetme varsayılan olarak atomiktir: içerik önce geçici bir dosyaya yazılır, ardından hedefin üzerine rename ile taşınır; böylece yazma sırasında oluşan bir kesinti mevcut dosyayı yarım bırakmaz. İsteğe bağlı olarak .bak yedeği alınabilir.

V. ROUND-TRIP DÖNÜŞÜM YAKLAŞIMI

A. Ayrıştırma modeli ve neyin korunduğu

ComposeMorph, belgeyi yaml-cpp'nin değiştirilebilir düğüm ağacına ayrıştırır. Bu bir CST değildir: kaynak metnin karakter aralıkları saklanmaz. Ağaçta korunan bilgi şunlardır: belge yapısı, eşlemelerde anahtar sırası, koleksiyonların akış/blok stili, skaler değerlerin metni, anchor ile paylaşılan düğümlerin kimliği ve — bu çalışma için kritik olan — skalerin kaynakta tırnaksız yazılıp yazılmadığını gösteren *belirtisiz etiket*: tırnaksız (plain) skalerler ?, tırnaklı ve blok (|, >) skalerler ise ! etiketi taşır. Ayrıştırmada kaybolan bilgi ise yorumlar, boş satırlar, girinti genişliği, tek/çift tırnak ayrımı, blok skaler stili ve anchor adlarıdır.

B. Değişiklik izleme ve çakışmalar

Kütüphane bir değişiklik günlüğü tutmaz. Her düzenleme, ilgili düğüme doğrudan yazar; dolayısıyla birleştirilecek iki sürüm ve çözülecek bir çakışma durumu oluşmaz. Modellenmemiş alanlar hiçbir aşamada okunmaz, kopyalanmaz veya yeniden inşa edilmez; kaydetmede ağaçta ne varsa o yazılır. Bilinmeyen özelliklerin ve x-* alanlarının korunması, ek bir mekanizmanın değil bu tasarım kararının sonucudur.

C. Serileştirme

yaml-cpp'nin varsayılan serileştiricisi bir skaleri tırnaksız yazmanın güvenli olup olmadığına yalnızca metnine bakarak karar verir: 0.8.0 sürümünde `IsValidPlainScalar` işlevi değeri null benzeri değerlere (null, ~, boş string) ve sözdizimsel olarak sakıncalı karakterlere karşı sınar, fakat boolean veya sayı gibi görünmesini dikkate almaz. Ayrıca ayrıştırmada üretilen ! etiketi serileştirmede yok sayılır. Sonuç, giriş bölümünde örneklenen davranıştır: `version: "3.8"` tırnaksız yazılır ve yeniden okunduğunda artık string değil kayan noktalı sayıdır.

Bu nedenle `ComposeMorph`, kaydetmeyi kendi ağaç yürütücüsü üzerinden yapar. Yürütücü, yaml-cpp'nin kendi düğüm serileştiricisiyle aynı çıktıyı üretir (aynı anchor/alias numaralandırması, aynı etiketler, aynı akış/blok stili, aynı anahtar sırası); tek farkı skaler yazma kararıdır:

1. **Birinci geçiş.** Ağaç dolaşarak her düğümün kaç kez referans edildiği sayılır. Bir düğüme birden çok yoldan ulaşılıyorsa o düğüm anchor gerektiriyor demektir. Kimlik karşılaştırması düğüm kimliği üzerinden yapılır; arama uzayını daraltmak için düğümler kaynaktaki konumlarına göre kovalara ayrılır.
2. **İkinci geçiş.** Ağaç yeniden dolaşılır ve yazılır. Birden çok referanslı bir düğüme ilk ulaşıldığında bir anchor atanır, sonraki ulaşımelerde alias yazılır. Eşlemeler ve diziler için düğümün stili (akış/blok) uygulanır.
3. **Skaler yazma kuralı.** Düğüm ! etiketi taşıyorsa (yani kaynakta tırnaklı ya da blok skaler olarak yazıldıysa) değer çift tırnakla yazılır; aksi hâlde yazma kararı yaml-cpp'nin varsayılan davranışına bırakılır. Böylece kaynakta tırnaksız yazılmış sayılar ve boolean'lar tırnaksız kalır, tırnaklı yazılmış string'ler string olarak kalır.

D. API üzerinden yazılan değerler

Round-trip sadakati, yalnızca dosyadan gelen değerleri kapsar. API üzerinden yazılan bir string de aynı tuzağa düşebilir: örneğin `setCommand({"run", "--workers", "4"})` çağrısında "4" tırnaksız yazılırsa `Compose` şeması `command` dizisinin tüm elemanlarının string olmasını beklediği için dosya reddedilir. Bu nedenle string alan her setter, yazdığı değeri bir belirsizlik sınavından geçirir: değer tırnaksız yazıldığında YAML 1.1 veya YAML 1.2 çekirdek şemasına göre null, boolean, tam sayı, kayan nokta, altmışlık sayı, zaman damgası ya da birleştirme anahtarı olarak çözümlenecekse düğüm ! ile işaretlenir ve yukarıdaki kural gereği tırnaklı yazılır. Sınama kasıtlı olarak geniş tutulmuştur: gereksiz tırnaklamanın bedeli estetik, eksik tırnaklamanın bedeli sessiz tip değişimidir.

Mevcut bir değer üzerine yazıldığında etiket korunduğu için, kaynakta tırnaklı olan bir alan güncellendikten sonra da tırnaklı kalır; kaynakta tırnaksız olan bir alan ise değeri belirsiz olmadıkça tırnaksız kalır.

E. Desteklenmeyen YAML özellikleri

Yorumlar ve boş satırlar ayrıştırmada kaybolduğu için çıktıda bulunmaz. Anchor/alias yapıları korunur — somut değerlere genişletilmez — ancak anchor adları ayrıştırıcı tarafından saklanmadığından çıktıda otomatik üretilmiş sayısal adlarla (&1, *1) yeniden yazılır. Tek tırnaklı skalerler çift tırnakla yazılır. Blok skalerlerin (|, >) içeriği korunur, ancak stil bilgisi saklanmadığından değer kaçış dizileri içeren tek satırlık çift tırnaklı bir string olarak yazılır (örneğin `"echo one\nnecho two\n"`); Veri Kümesi B'deki 647 dosyanın 55'inde (%8,5) en az bir blok skaler bulunmaktadır. Bu davranışlar ölçümleriyle birlikte bulgular (Bölüm VIII.) ve kısıtlamalar (Bölüm XI.) bölümlerinde raporlanmaktadır.

VI. GERÇEKLEŞTİRİM

ComposeMorph C++20 ile yazılmıştır ve GCC 11 veya Clang 13 ve üzeri derleyicilerle kurulabilir; değerlendirmede GCC 14.2 kullanılmıştır. Derleme sistemi CMake (≥ 3.16), tek dış bağımlılık yaml-cpp'dir ($\geq 0.7.0$; ölçümlerde 0.8.0). Birim testleri GoogleTest ile yazılmış ve CTest'e bağlanmıştır; paket şu anda 55 test içermektedir.

Kütüphane kodu toplam 2.471 satırdır (1.832 satır .cpp kaynak kodu ve 639 satır başlık). Değerlendirme altyapısı ayrıca beş adet C++ ölçüm aracı (toplam 557 satır) ve on iki Python betiği (toplam yaklaşık 4.140 satır) içerir. Depoda src/ ve include/ kütüphaneyi, benchmarks/ ölçüm araçlarını, datasets/ iki veri kümesini, scripts/ deney sürücülerini, results/ ham çıktıları, tabloları ve figürleri, paper/ ise bu bildiriyi barındırır.

Docker Compose ile bütünleşme iki düzeydedir. Kütüphane içinde ComposeFile::validate() yaygın alanlar için iş kuralı denetimleri yapar: servis adının biçimini ve servis düğümünün eşleme olmasını; extends kullanan servisler dışında image ya da build alanlarından en az birinin bulunmasını; ports, volumes, networks ve environment gibi temel alanların YAML türlerini; port ve volume girdilerinin biçimini ve yinelenmesini; liste biçimli environment girdilerindeki ve networks başvurularındaki tekrarları ve üst seviyede networks bölümü bulunan dosyalarda tanımsız ağlara yapılan başvuruları denetler. İnterpolasyon içeren değerler (örneğin "\${PORT}:80") çalışma zamanında çözüldüğünden denetim dışında bırakılır. Asıl doğrulama ise dış araçla yapılır: üretilen her çıktı docker compose config komutundan geçirilir (Bölüm VII.).

VII. DENEYSEL YÖNTEM

A. Araştırma soruları

- **RQ1 (Doğruluk):** Kütüphane gerçek Compose dosyalarını ayrıştırıp değiştirdikten sonra geçerli Compose dosyaları üretiyor mu?
- **RQ2 (Koruma):** Yalnızca belirli bir alan değiştirildiğinde, değişiklik ile ilgisi olmayan yapılandırma bilgisinin ne kadarı korunuyor?
- **RQ3 (Değişiklik yerelliği):** Hedeflenen alan değiştirilirken dosyada ne kadar gereksiz değişiklik oluşuyor?
- **RQ4 (İleriye dönük uyumluluk):** Nesne modelinde tanımlanmamış Compose özellikleri kaybedilmeden taşıyor mu?
- **RQ5 (Performans):** Yükleme, değiştirme ve serileştirme maliyeti gerçek kullanım için kabul edilebilir mi?

B. Veri kümeleri

Veri Kümesi A (kontrollü), elle yazılmış 38 dosyadan oluşur: 24 dosya Compose özelliklerini tek tek (build, kısa ve uzun sözdizimli environment/ports/volumes, networks, depends_on, healthcheck, deploy, configs, secrets, labels, profiles, logging, sysctls, ulimits, devices, x-* alanları, modellenmemiş alanlar ve iç içe genel özellikler) izole eder; 2 dosya bunları birlikte içerir; 10 dosya sınır durumlarını (anchor/alias ve birleştirme anahtarı, tırnaklı skaler biçimleri, yalnızca build içeren servis, Unicode içerik, null ve boş değerler, external kaynaklar, extends, include, yalnızca büyük/küçük harfle ayrıışan anahtarlar) sınar; 2 dosya ise koruma deneylerinin işaretçi kalıplarını taşır.

Veri Kümesi B (gerçek dünya), açık kaynak GitHub projelerinden iki adımda otomatik olarak toplanmış 647 Compose dosyasından oluşur. İlk adımda GitHub kod

arama API'sinin `filename:docker-compose.yml` sorgusundan 597, depo aramasının `topic:docker-compose` sorgusundan 3 dosya alınmıştır; adaylar depo ve dosya yolu düzeyinde tekilleştirilmiş, yalnızca geçerli YAML olarak ayrıştırılan ve bir `services` eşlemesi içeren dosyalar tutulmuştur. Bu dosyaların satır sayısı medyanı 36, en büyüğü ise 405 satırdır; büyük dosya grupları hiç temsil edilmemektedir. Bu nedenle ikinci adımda, çok servisli yığınlar barındırma olasılığı yüksek depolarda (`homelab`, `self-hosted`, izleme yığınları) tüm dosya ağacı taranarak 500 satırın üzerindeki 47 Compose dosyası ayrıca eklenmiştir. Dolayısıyla korpus büyük dosyalar yönünden bilinçli olarak zenginleştirilmiştir. Toplama ve analiz betikleri depoda yayımlanmıştır. Birleşik korpus yine de literatürdeki gözlemlerle uyumlu biçimde küçük dosyalara eğilimlidir: satır sayısı medyanı 39 (en büyük dosya 2.591 satır), servis sayısı medyanı 2 (en fazla 56), üst seviye bölüm sayısı medyanı 2, YAML iç içelik derinliği medyanı 4 ve `x-*` alanı sayısı medyanı 0'dır. Son değer, `x-*` ve modellenmemiş alan deneylerinde neden kontrollü işaretçi enjeksiyonuna başvurduğumuzu açıklar: bu alanlar gerçek dosyalarda doğal olarak nadirdir.

C. Deneyler

Yedi deney yürütülmüştür: (1) kimlik round-trip (değişiklik yapmadan yükle-kaydet), (2) hedefli değişiklik ve değişiklik yerelliği (on ayrı özellik üzerinde), (3–4) modellenmemiş alan ve `x-*` alan koruma, (5) `docker compose config` ile doğrulama, (6) ham `yaml-cpp` ile karşılaştırma, (7) performans. Her deney her iki veri kümesi üzerinde de koşulmuştur. Deney 6'da üç araç karşılaştırılır: `ComposeMorph`; düğümü yerinde değiştiren *dikkatli* ham `yaml-cpp` kodu; ve aynı düzenlemeyi ilgili alt ağacı yeniden inşa ederek yapan *saf* ham `yaml-cpp` kodu. Deney 2'de kütüphane `ports`, `extra_hosts` ve `networks` için yerinde değiştirme değil yalnızca ekleme ve çıkarma işlemi sunduğundan, bu üç özellikteki hedefli değişiklik listeye tek bir girdi eklemektir. `volume-source` düzenlemesinde en az bir kısa sözdizimli `volume` girdisi bulunan servisler uygun kabul edilmiş, `volume` girdilerinin tamamı uzun sözdizimli olan servisler deneyden dışlanmıştır. Dikkatli araç `yaml-cpp`'nin varsayılan serileştiricisini kullandığından, Deney 5 ve tırnak analizinde aynı zamanda “düzeltme öncesi” sütununu sağlar; bu eşdeğerlik ölçülmüştür (Bölüm VIII.).

D. Metrikler ve istatistiksel değerlendirme

Metinsel koruma, girdi ile çıktı arasındaki değişen satır sayısı, değişen satır oranı, satır düzeyinde Levenshtein uzaklığı ve normalize edilmiş düzenleme uzaklığı ile ölçülür. Değişiklik yerelliği için *beklenen değişen satır / gerçekleşen değişen satır* oranı kullanılır; beklenen değer tek alanlık bir düzenleme için 1'dir. Aynı dosyanın değişiklik yapılmadan kaydedilmesindeki taban gürültü, düzeltilmiş oranda gerçekleşen satır sayısından düşülür. Anlamsal koruma iki bağımsız doğrulama mekanizmasıyla ölçülür: `docker compose config` (Compose şemasının kendi doğrulayıcısı) ve PyYAML ile yeniden yükleme (değerlerin YAML tiplerinin korunup korunmadığı). Performans ölçümleri her dosya için 30 yinelemeyle yapılır; medyan, ortalama, standart sapma ve p95 raporlanır. İki serileştirici karşılaştırılırken ölçümler aynı süreçte, aynı ağaç üzerinde ve yineleme başına sıra değiştirerek eşleşik biçimde alınır; dağılım varsayımı yapmamak için Wilcoxon işaretli sıralar testi (normal yaklaşım) kullanılır.

E. Ortam ve tekrarlanabilirlik

Ölçümler 12. nesil Intel Core i7-1255U (12 mantıksal çekirdek), 15,3 GiB RAM, Debian GNU/Linux 13 (çekirdek 6.12), GCC 14.2.0, Release derleme, C++20 ve Docker Compose 2.26.1 ortamında yapılmıştır. Tüm deneyler depodaki tek bir komutla yeniden üretilebilir: `./scripts/run-all-experiments.sh`. Ham çıktılar, tablolar ve figürler `results/` altında yayımlanmıştır; düzeltme öncesi tablolar karşılaştırma için `results/pre-fix/` al-

Tablo I. Docker Compose doğrulama başarı oranı (Veri Kümesi B)

Çıktı	ComposeMorph	Karşılaştırma sürümü
Kimlik round-trip	491/491 (%100)	187/491 (%38,1)
Hedefli image düzenlemesi	401/401 (%100)	153/401 (%38,2)
Toplam	892/892 (%100)	340/892 (%38,1)

tında saklanmıştır.

VIII. BULGULAR

Bulgular, deney çıktılarından otomatik olarak üretilen özetlere dayanmaktadır. “Karşılaştırma sürümü” sütunları, aynı düzenlemeleri yaml-cpp’nin varsayılan serileştiricisiyle gerçekleştiren dikkatli ham yaml-cpp aracına aittir. Bu araç, düzeltme öncesi ComposeMorph ile 100 dosyanın tamamında bayt düzeyinde aynı çıktıyı üretmiştir.

A. RQ1 — Doğruluk

Veri Kümesi B’deki 647 dosyanın tamamı ayrıştırılmış (647/647), kaydedilmiş ve kaydedilen çıktı yeniden ayrıştırılabilmiştir (647/647). Veri Kümesi A’da da 38/38 dosya için aynı sonuç alınmıştır.

Asıl ayrım dış doğrulamada ortaya çıkar. Korpustaki 647 dosyanın 491’i (%75,9) docker compose config tarafından zaten geçerli kabul edilmektedir; kalanlar eksik .env dosyaları, tanımsız interpolasyon değişkenleri veya bozuk sözdizimi gibi kütüphaneden bağımsız nedenlerle reddedilir ve aşağıdaki oranların dışında tutulur. Tablo I, bu geçerli girdilerden üretilen çıktıların doğrulama sonuçlarını göstermektedir.

Karşılaştırma sürümünde geçersizleşen 552 çıktının 548’i, tırnaklı bir skalerin tırnaksız yazılması sonucu string olması gereken bir alanın sayıya veya boolean’a dönüşmesinden kaynaklanmaktadır. Kalan 4 vakada çıktı, Docker’ın YAML ayrıştırıcısı için sözdizimsel olarak dahi geçerli değildir: kaynakta " : 80 " biçiminde tırnaklanmış bir değer, akış dizisi içinde tırnaksız yazıldığında PyYAML ve go-yaml tarafından okunamamaktadır. ComposeMorph’un tırnak koruyan serileştiricisiyle bu vakaların tamamı ortadan kalkmıştır. Veri Kümesi A’da da ComposeMorph çıktılarının tamamı geçerlidir (64/64); karşılaştırma sürümünde yalnızca tırnaklı skaler biçimlerini sınavan dosyanın iki çıktısı reddedilmektedir (62/64).

B. RQ2 — Koruma

Koruma üç ayrı düzeyde ölçülmüştür.

Yapısal koruma. Gerçek dosyalara kontrollü olarak eklenen ve kütüphanenin modellemediği alanlar, ilgisiz bir alan değiştirildikten sonra tüm vakalarda korunmuştur: 802/802 modellenmemiş alan ve 1.203/1.203 x^* alanı örneği (Veri Kümesi A’da 96/96 ve 144/144). Kontrollü sınır durumlarının altısı da (null değerli x^* , skaler değerli x^* , liste değerli x^* , liste biçiminde bilinmeyen üst seviye bölüm, tipli bir bölümün içine gömülü bilinmeyen alan, başka bir bilinmeyen bloğun içine iki düzey gömülü x^*) korunmuştur.

Değer tipleri. Korpustaki 617 dosyada açıkça tırnaklanmış 9.807 skaler değer tespit edilmiştir. ComposeMorph ile round-trip sonrasında bunların hiçbiri tip değiştirmemiştir. Karşılaştırma sürümünde ise 452 dosyada (tırnaklı skaler içeren dosyaların %73,3’ü) 2.570 değer tip değiştirmektedir: tam sayı görünümlü değerlerin %99,9’u, kayan nokta görünümlülerin %100’ü ve boolean kelimelerinin %99,5’i. Buna bağlı olarak, çıktının girdiyle PyYAML’a göre birebir aynı veriye çözümleme oranı 33/100’den 100/100’e çıkmaktadır.

Metinsel koruma. Buna karşılık çıktı bayt düzeyinde girdinin aynısı değildir: değişiklik yapılmadan kaydedilen dosyaların yalnızca %4,0'ü (26/647) özdeştir; değişen satır oranının medyanı 0,167'dir (düzeltme öncesi %0,77 ve 0,250). Satır düzeyinde Levenshtein uzaklığının medyanı 7, normalize edilmiş düzenleme uzaklığının medyanı 0,167'dir. Kalan farklar yorumların ve boş satırların kaybindan, girinti normalizasyonundan, tek tırnağın çift tırnağa ve blok skalerlerin tek satırlık çift tırnaklı string'e dönüşmesinden kaynaklanır. Anchor/alias ve birleştirme anahtarı yapıları somut değerlere genişletilmeden korunur, ancak anchor adları sayısal adlarla yeniden yazılır.

C. RQ3 — Değişiklik Yerelliği

On ayrı özellik üzerinde 1.467 düzenleme senaryosu koşulmuştur. Tek alanlık bir düzenlemede değişen satır sayısının medyanı port için 8; image, env ve volume-source için 9; restart için 10; healthcheck-retries için 13; network için 14; deploy-cpus için 20; hostname için 23 ve extra-host için 36'dır. Ham yerellik oranının medyanı bu nedenle 0,03–0,13 arasındadır. Bu satırların neredeyse tamamı düzenlemenin kendisinden değil, kaydetmede yorum ve boş satır kaybindan ve girinti normalizasyonundan doğan taban gürültüden kaynaklanır. Aynı dosyanın değişiklik yapılmadan kaydedilmesindeki gürültü düşüldüğünde, on özelliğin tamamında düzeltilmiş yerellik oranının medyanı 1,0'dır: düzenlemenin kendi katkısı tek satırla sınırlıdır. Buna karşın kod incelemesinde görülen ham diff taban gürültüyü de içerdiğinden tek satırlık değildir (Bölüm IX.).

Aynı ölçüm, kısa sözdizimi hatasının etkisini de ortaya koyar. Düzeltme öncesinde, liste biçimli environment bölümüne sahip 94 servisin 83'ünde düzenleme, gürültü düşüldükten sonra da birden fazla satırı değiştirmekteydi (medyan 3 satır), çünkü tek bir değişkenin güncellenmesi listenin tamamını eşlemeye dönüştürüyordu; extra_hosts'a girdi eklemede bu sayı 9/34'tü. Düzeltmeden sonra her iki özellikte de böyle bir vaka kalmamıştır (0/94 ve 0/34). Kısa sözdizimi hatasından etkilenmeyen image düzenlemesinde bu sayı önce de sonra da sıfırdır.

Deneyde başarısızlık gözlenmemiştir: on özelliğin tamamında 1.467 düzenlemenin tümü tamamlanmıştır.

D. RQ4 — İleriye Dönük Uyumluluk

Kütüphanenin nesne modelinde bulunmayan alanlar, hem x- önekli x-* alanları hem de gelecekteki spesifikasyon alanlarını temsil eden x- öneksiz modellenmemiş alanlar için tam olarak korunmuştur (RQ2). Ancak koruma ile geçerlilik aynı şey değildir. Yalnızca x-* alanları eklenen dosyaların tamamı docker compose config tarafından kabul edilirken (83/83; temel çizgide 33/83), x- öneksiz bilinmeyen alan eklenen dosyaların hiçbirisi kabul edilmemiştir (0/83). Bunun nedeni kütüphane değil, Compose şemasının tanınmayan anahtarlara yalnızca x- öneki taşıdıklarında izin vermesidir. Pratik sonuç şudur: ileriye dönük uyumluluk için ayrılmış resmî mekanizma x-* alanlarıdır ve kütüphane bu mekanizmayı eksiksiz desteklemektedir.

E. RQ5 — Performans

Tablo II, dosya boyutuna göre yükle-değiştir-kaydet sürelerinin medyanlarını vermektedir. Ölçümler, her boyut grubunda alfabetik sıradaki ilk 20 dosyadan image düzenlemesine uygun bir servis içerenler üzerinde yapılmıştır. Değiştirme adımının maliyeti diğer iki adımın yanında ihmal edilebilir düzeydedir; toplam süre en büyük dosyalarda bile onlarca milisaniye mertebesinde. Tepe bellek kullanımı 4,7 MiB ile 8,2 MiB arasındadır.

Tırnak koruyan serileştiricinin bir maliyeti olup olmadığını ölçmek için iki serileştirici aynı süreçte, aynı ağaç üzerinde ve yineleme başına sıra değiştirilerek karşılaştırılmıştır. ComposeMorph'un serileştiricisi tüm boyut gruplarında yamll-cpp'nin kendi serileştiricisinden hızlıdır: medyan oran küçük ve büyük dosyalarda 0,96, orta dosyalarda 0,95, çok büyük dosya-

Tablo II. Yükle-değiştir-kaydet süreleri, medyan, ms (Veri Kümesi B'den 59 dosya: 17/20/20/2; dosya başına 30 yinleme)

Grup	Yükle	Değiştir	Kaydet	Toplam
Küçük (<100 satır)	0,15	0,003	0,15	0,31
Orta (100–500)	0,64	0,007	0,55	1,21
Büyük (500–2000)	2,25	0,019	1,96	4,26
Çok büyük (>2000)	8,62	0,076	6,17	15,23

larda 0,82'dir ve fark her grupta istatistiksel olarak anlamlıdır (Wilcoxon işaretli sıralar testi, $p < 10^{-10}$). Her dosyanın ilk yinlemesi soğuk başlangıç maliyetini içerir: sayfa hataları, dosya sistemi önbelleği ve API üzerinden yazılan değerlerin belirsizlik sınavında kullanılan düzenli ifadenin süreç başına bir kez derlenmesi. İlk yinlemede değiştirme adımının medyanı 0,80 ms'dir (59 dosyada en fazla 1,07 ms). Bu yinlemeler istatistiklerden çıkarılmamıştır; bu nedenle ortalama ve p95 değerleri yukarı çekilmiştir ve tipik maliyeti medyan daha iyi temsil eder.

F. Ham yaml-cpp ile karşılaştırma

Deney 6, ComposeMorph'u iki ham yaml-cpp kullanımıyla karşılaştırır. Serileştirme düzeyinde ComposeMorph ile dikkatli ham kullanım arasındaki çıktı farkı 100 dosyanın 96'sında yalnızca tırnak karakterlerinden ibarettir, 4'ünde çıktı zaten özdeştir; tırnak dışında farklılık yoktur. Düzenleme güvenliği düzeyinde ise fark büyüktür (Şekil 1): alt ağacı yeniden inşa eden saf kullanım, ölçülebilen hiçbir vakada düzenlenen servisin diğer alanlarını koruyamamıştır (image için 0/149, hostname için 0/39, env için 0/116), ComposeMorph ise tümünü korumuştur (149/149, 39/39, 116/116). Aynı durum işaretçi alanlarında da görülür: saf kullanım servis düzeyindeki 200 işaretçinin tamamını kaybederken üst seviyedekilerin tamamını korur; hasar, üzerine yazılan alt ağaçla sınırlıdır.

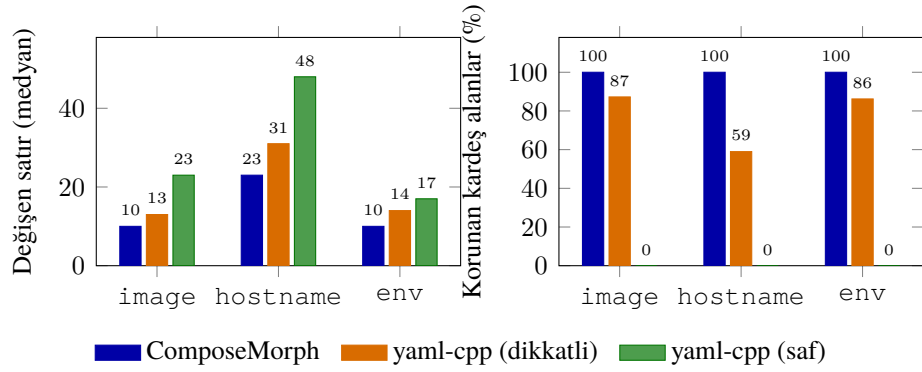
Şekil 1, dikkatli ham kullanımın da kardeş alanları her vakada korumadığını göstermektedir (130/149, 23/39, 100/116). Bu fark yapısal bir kayıp değildir. Korunma, kardeş değerlerin düzenleme öncesi ve sonrasında PyYAML ile karşılaştırılmasıyla ölçülür; dikkatli araç yaml-cpp'nin varsayılan serileştiricisini kullandığından tırnaklı kardeş değerler (ör. "8022 : 22") tip değiştirir. Bu 51 vakanın 48'inde kardeş alanların tümü çıktıda yerindedir ve yalnızca skaler gösterimleri değişmiştir; kalan 3 vakada çıktı, RQ1 altında açıklanan " : 80 " sorunu nedeniyle ayrıştırılamaz hâle gelmiştir.

Ergonomi açısından, ComposeMorph'ta on düzenlemenin her biri tek satırdır; aynı düzenlemeleri ham yaml-cpp ile doğru yapmak 1 ile 24 satır arasında kod gerektirir (volume-source için 24, extra-host için 22, env için 18 satır). Satır sayısı hatalı kodu ayırt etmez: env düzenlemesinde bölümün tamamını yeniden kuran saf sürüm (6 satır), doğru sürümün üçte biri uzunluğundadır. image ve hostname için ise doğru sürüm tek satırdır ve saf sürüm (5 ve 3 satır) daha uzundur.

IX. TARTIŞMA

A. Tırnak, biçim değil anlamdır

Tırnak kaybı kozmetik değildir: geçerli girdilerden üretilen çıktıların %62'si Docker tarafından reddedilmiştir. Ayrıştırıcı tırnak bilgisini ! etiketiyle sakladığından düzeltme, mevcut etiketin serileştirmede kullanılmasını gerektirir. PyYAML ve Docker bazı skalerleri farklı yorumladığı için iki bağımsız doğrulama mekanizması kullanılmıştır.



Şekil 1. ComposeMorph ile iki ham yamll-cpp kullanımının karşılaştırması (Veri Kümesi B). Solda düzenleme başına değişen satır sayısının medyanı, sağda düzenlenen servisin *diğer* alanlarının korunma oranı. Alt ağacı yeniden inşa eden saf kullanım hiçbir vakada kardeş alanları koruyamamaktadır (%0, çubuk görünmez). Dikkatli kullanımdaki eksik, tırnak kaybı sonucu tip değişiminden kaynaklanır; kardeş alanlar yapısal olarak yerindedir.

B. Hangi senaryoda ne beklenmeli

ComposeMorph tek alanlı otomasyon değişikliklerinde bilinmeyen alanları korur ve geçerli çıktı üretir. Yorum, boşluk, girinti ve blok skalerlerin metinsel biçiminin korunması gerekiyorsa CST tabanlı bir araç gerekir.

C. Performans ile koruma arasında bir ödünleşme yok

Koruma için performans bedeli ölçülmemiştir: tırnak koruyan serileştirici tüm boyut gruplarında yamll-cpp'den hızlı, düzenleme maliyeti ise yükleme ve kaydetmenin yanında ihmal edilebilir düzeydedir.

D. Compose'a özgü API neden fark yaratıyor?

Doğru yazılmış ham yamll-cpp kodu aynı yapısal korumayı sağlayabilir. ComposeMorph'un katkısı, hedef düğümü yerinde değiştirmeyi ve kısa sözdizimini korumayı varsayılan kullanım hâline getirerek sessiz alan kaybı riskini azaltmasıdır.

X. GEÇERLİLİK TEHDİTLERİ

A. İç Geçerlilik

Deney altyapısı kendi hatalarını ölçümlere taşıyabilir; ilk koşullarda interpolasyonlu imaj adlarını bozan bir ölçüm hatası saptanıp düzeltilmiştir. Koruma deneylerindeki bilinmeyen ve $\times$ -* alanlar sentetiktir; sonuçlar bu kalıplar için geçerlidir. Saf ham yamll-cpp karşılaştırması da gerçek projelerdeki tüm hatalı kullanımları temsil etmez.

B. Dış Geçerlilik

GitHub'dan toplanan 647 dosya özel ve kurum içi kullanımları temsil etmeyebilir. Korpus küçük dosyalara eğilimlidir [2, 22]; büyük dosyalar ayrıca eklendiği hâlde 2.000 satırın üzerindeki performans sonuçları yalnızca iki dosyaya dayanır.

C. Yapı Geçerliliği

Değişen satır sayısı anlamsal korumayı tek başına göstermez. Bu nedenle metinsel ölçümler docker compose config ve PyYAML doğrulamasıyla birlikte raporlanmıştır. PyYAML ile go-yaml bazı değerleri farklı çözümlediğinden sonuçlar ayrı tutulmuştur.

D. Sürüm Geçerliliği

Sonuçlar yamll-cpp 0.8.0, Docker Compose 2.26.1 ve ölçüm tarihindeki spesifikasyona bağlıdır. Şema veya serileştirici değiştiğinde oranlar da değişebilir; deneyler bu nedenle tek komutla yeniden çalıştırılabilir.

XI. KISITLAMALAR

Aşağıdaki sınırlar ölçülmüş ve gizlenmemiştir.

- **Yorum satırları korunmaz.** Yorumlar yamll-cpp'nin ayrıştırıcısında veri modeline hiç taşınmadığı için çıktıda bulunmaz. Bu, kütüphanenin düzeltebileceği bir davranış değildir; altta yatan kütüphanede yorum koruma talebi 2015'ten beri açıktır [8].
- **Round-trip bayt düzeyinde özdeş değildir.** Boş satırlar ve girinti tercihleri korunmaz, tek tırnak çift tırnağa dönüşür; dosyaların yalnızca %4,0'ü değişiklik yapılmadan kaydedildiğinde özdeş kalır. Değerlerin YAML tipleri ise korunur.
- **Blok skaler stili kaybolur.** | ve > ile yazılmış çok satırlı değerlerin içeriği korunur, ancak kaçış dizileri içeren tek satırlık çift tırnaklı string olarak yazılır. Bu, çok satırlı `command` veya sağlık kontrolü betiklerinin okunabilirliğini düşürür; Veri Kümesi B'deki dosyaların %8,5'i (55/647) blok skaler içermektedir.
- **Anchor adları kaybolur.** Anchor/alias ve birleştirme anahtarı yapıları genişletilmeden korunur, fakat adlar ayrıştırıcıda saklanmadığı için çıktıda `&1`, `*1` biçiminde yeniden üretilir.
- **x- öneksiz bilinmeyen alanlar şemaya aykırıdır.** Kütüphane bu alanları eksiksiz korur, ancak Compose şeması yalnızca x- öneklili alanlara izin verdiğinden dosya `docker compose config` tarafından reddedilir. Bu, kütüphaneden bağımsız bir spesifikasyon kısıtıdır.
- **Doğrulama tam Compose şemasını kapsamaz.** `validate()` yaygın alanları ve iş kurallarını denetler, ancak Compose şemasındaki bütün alanları ve sürüme bağlı bütün kuralları eksiksiz doğrulamaz; tam şema doğrulaması dış araca, yani `docker compose config` komutuna bırakılmıştır.
- **Platform.** Geliştirme ve ölçümler Linux üzerinde yapılmıştır; Windows ve macOS'ta test edilmemiştir.

XII. SONUÇ

Docker Compose dosyalarının programatik olarak değiştirilmesi, YAML sözdizimini ayrıştırmaktan ibaret değildir: dosyada aracın modellemediği alanlar, kullanıcı tanımlı x-* alanları, farklı sözdizimi gösterimleri ve anlam taşıyan tırnak tercihleri bulunur. Bu bildiride, problemi C++ tarafında ele alan ComposeMorph kütüphanesi sunulmuş; yaklaşımın sağladığı olanaklar ve sınırları 647 gerçek dünya ve 38 kontrollü Compose dosyası üzerinde yürütülen yedi deneyle ölçülmüştür.

Sonuçlar üç noktada toplanabilir. Birincisi, düzenlemeyi düğüm ağacında yerinde yapmak, modellenmemiş alanların korunması için yeterlidir: dosyalara eklenen modellenmemiş alanların ve x-* alanlarının tamamı korunmuştur. İkincisi, düğüm ağacı tabanlı serileştirme sessiz bir tip değişimi kusuru barındırır; ayrıştırıcının sakladığı tırnak bilgisinin serileştirmede kullanılması bu kusuru ortadan kaldırmış ve Docker doğrulama başarısını %38,1'den %100'e çıkarmıştır;

üstelik bunun bir performans bedeli yoktur. Üçüncüsü, koruma davranışı doğru yazılmış ham yaml-cpp koduyla da elde edilebilir; tipli API'nin katkısı, doğru olanı varsayılan yol hâline getirmesidir — ölçümlerimizde makul görünen ama hatalı bir ham kullanım, düzenlenen servisin diğer alanlarını hiçbir vakada koruyamamıştır.

Gelecek çalışmalar için üç yön öne çıkmaktadır. Birincisi, yorum ve boşluk korumasını da kapsayan tam CST tabanlı düzenleme; bu, diğer ekosistemlerde çözülmüş olan problemin C++ tarafındaki karşılığıdır. İkincisi, mevcut doğrulama kapsamının Compose şemasındaki daha fazla alan ve kuralı kapsayacak biçimde genişletilmesi. Üçüncüsü, nesne modelinin Compose Specification ile otomatik eşitlenmesi ve aynı yaklaşımın Kubernetes manifestoları gibi diğer yapılandırma biçimlerine uygulanması.

KAYNAKLAR

- [1] “The Compose specification,” <https://github.com/compose-spec/compose-spec>, 2026.
- [2] M. H. Ibrahim, M. Sayagh ve A. E. Hassan, “A study of how Docker Compose is used to compose multi-component systems,” *Empirical Software Engineering*, vol. 26, 2021. <https://doi.org/10.1007/s10664-021-10025-1>
- [3] J. Beder ve katkıda bulunanlar, “yaml-cpp: A YAML parser and emitter in C++,” <https://github.com/jbeder/yaml-cpp>, 2026.
- [4] A. van der Neut, “ruamel.yaml: YAML 1.2 loader/dumper package for Python,” <https://pypi.org/project/ruamel.yaml/>, 2026.
- [5] E. Aro, “yaml: YAML parser and serialiser for JavaScript,” <https://github.com/eemeli/yaml>, 2026.
- [6] “compose-go: Reference library for parsing and loading Compose YAML files,” <https://github.com/compose-spec/compose-go>, 2026.
- [7] “kyaml: YAML processing for Kubernetes resource configuration,” <https://pkg.go.dev/sigs.k8s.io/kustomize/kyaml>, 2026.
- [8] “option to not eat comments (#154),” <https://github.com/jbeder/yaml-cpp/issues/154>, yaml-cpp hata kayıt sistemi, 2015.
- [9] “Preserve comments after parsing and re-emit them (#457),” <https://github.com/jbeder/yaml-cpp/issues/457>, yaml-cpp hata kayıt sistemi, 2026.
- [10] “libyaml: A C library for parsing and emitting YAML,” <https://github.com/yaml/libyaml>, 2026.
- [11] “Support for emitting and parsing comments (#42),” <https://github.com/yaml/libyaml/issues/42>, libyaml hata kayıt sistemi, 2016.
- [12] J. P. Magalhães ve katkıda bulunanlar, “rapidyaml (ryml): A library to parse and emit YAML, fast,” <https://github.com/biojppm/rapidyaml>, 2026.
- [13] “Preserving scalar type information when emitting (#30),” <https://github.com/biojppm/rapidyaml/issues/30>, rapidyaml hata kayıt sistemi, 2020.

- [14] “fkYAML: A C++ header-only YAML library,” <https://fktn-k.github.io/fkYAML/>, 2026.
- [15] “yaml-edit: A formatting-preserving YAML editor for Rust,” https://docs.rs/yaml-edit/latest/yaml_edit/, 2026.
- [16] Instagram Engineering, “LibCST: A concrete syntax tree parser and serializer library for Python,” <https://github.com/Instagram/LibCST>, 2026.
- [17] D. Quirl, “cstree: Concrete syntax tree library,” <https://github.com/domenicquir1/cstree>, 2026.
- [18] “Preserving comments and other human-readable syntax in output yaml (#259),” <https://github.com/kubernetes-sigs/kustomize/issues/259>, kustomize hata kayıt sistemi, 2026.
- [19] T. Tosi, “fishfinger: A Go docker-compose programmatic library,” <https://github.com/TimTosi/fishfinger>, 2026.
- [20] “compose-py: Python library for parsing and loading compose files,” <https://github.com/bonprosoft/compose-py>, 2026.
- [21] “docker-compose-yaml-parser: Parse and modify docker-compose.yaml files,” <https://github.com/WisdomSky/docker-compose-yaml-parser>, 2026.
- [22] K. Eng, A. Hindle ve E. Stroulia, “Patterns of multi-container composition for service orchestration with Docker Compose,” *Empirical Software Engineering*, vol. 29, no. 3, makale 65, 2024. <https://doi.org/10.1007/s10664-024-10462-8>
- [23] I. Predoaia, D. Kolovos, A. Garcia-Dominguez, M. Lenk, W. Ebel ve J. Burkl, “Towards processing YAML documents with model management languages,” *Companion Proceedings of MODELS '24* içinde, Linz, Avusturya, 2024.